Reference

Contents

- htmx Core Attributes
- htmx Additional Attributes
- htmx CSS Classes
- htmx Request Headers
- htmx Response Headers
- htmx Events
- htmx Extensions
- JavaScript API
- Configuration Options

Core Attribute Reference

The most common attributes when using htmx.

Attribute	Description
`hx-get`	issues a `GET` to the specified URL
`hx-post`	issues a `POST` to the specified URL
`hx-on*`	handle events with inline scripts on elements
`hx-push-url`	push a URL into the browser location bar to create history
`hx-select`	select content to swap in from a response
`hx-select-oob`	select content to swap in from a response, somewhere other than the target (out of band)
`hx-swap`	controls how content will swap in (`outerHTML` , `beforeend` , `afterend` , ...)
`hx-swap-oob`	mark element to swap in from a response (out of band)
`hx-target`	specifies the target element to be swapped

Attribute	Description
`hx-trigger`	specifies the event that triggers the request
`hx-vals`	add values to submit with the request (JSON format)

Additional Attribute Reference

All other attributes available in htmx.

Attribute	Description
`hx-boost`	add progressive enhancement for links and forms
`hx-confirm`	shows a `confirm()` dialog before issuing a request
`hx-delete`	issues a `DELETE` to the specified URL
`hx-disable`	disables htmx processing for the given node and any children nodes
`hx-disabled-elt`	adds the `disabled` attribute to the specified elements while a request is in flight
`hx-disinherit`	control and disable automatic attribute inheritance for child nodes
`hx-encoding`	changes the request encoding type
`hx-ext`	extensions to use for this element
`hx-headers`	adds to the headers that will be submitted with the request
`hx-history`	prevent sensitive data being saved to the history cache
`hx-history-elt`	the element to snapshot and restore during history navigation
`hx-include`	include additional data in requests
`hx-indicator`	the element to put the `htmx-request` class on during the request
`hx-inherit`	control and enable automatic attribute inheritance for child nodes if it has been disabled by default
`hx-params`	filters the parameters that will be submitted with a request
`hx-patch`	issues a `PATCH` to the specified URL
`hx-preserve`	specifies elements to keep unchanged between requests
`hx-prompt`	shows a `prompt()` before submitting a request
`hx-put`	issues a `PUT` to the specified URL

Attribute	Description
`hx-replace-url`	replace the URL in the browser location bar
`hx-request`	configures various aspects of the request
`hx-sync`	control how requests made by different elements are synchronized
`hx-validate`	force elements to validate themselves before a request
`hx-vars`	adds values dynamically to the parameters to submit with the request (deprecated, please use `hx-vals`)

CSS Class Reference

Class	Description
`htmx-added`	Applied to a new piece of content before it is swapped, removed after it is settled.
`htmx-indicator`	A dynamically generated class that will toggle visible (opacity:1) when a `htmx-request` class is present
`htmx-request`	Applied to either the element or the element specified with `hx-indicator` while a request is ongoing
`htmx-settling`	Applied to a target after content is swapped, removed after it is settled. The duration can be modified via `hx-swap` .
`htmx-swapping`	Applied to a target before any content is swapped, removed after it is swapped. The duration can be modified via `hx-swap` .

HTTP Header Reference

Request Headers Reference

Header	Description
`HX-Boosted`	indicates that the request is via an element using hx-boost
`HX-Current-URL`	the current URL of the browser
`HX-History-Restore-Request`	“true” if the request is for history restoration after a miss in the local history cache
`HX-Prompt`	the user response to an hx-prompt

Header	Description
`HX-Request`	always “true”
`HX-Target`	the `id` of the target element if it exists
`HX-Trigger-Name`	the `name` of the triggered element if it exists
`HX-Trigger`	the `id` of the triggered element if it exists

Response Headers Reference

Header	Description
`HX-Location`	allows you to do a client-side redirect that does not do a full page reload
`HX-Push-Url`	pushes a new url into the history stack
`HX-Redirect`	can be used to do a client-side redirect to a new location
`HX-Refresh`	if set to “true” the client-side will do a full refresh of the page
`HX-Replace-Url`	replaces the current URL in the location bar
`HX-Reswap`	allows you to specify how the response will be swapped. See hx-swap for possible values
`HX-Retarget`	a CSS selector that updates the target of the content update to a different element on the page
`HX-Reselect`	a CSS selector that allows you to choose which part of the response is used to be swapped in. Overrides an existing `hx-select` on the triggering element
`HX-Trigger`	allows you to trigger client-side events
`HX-Trigger-After-Settle`	allows you to trigger client-side events after the settle step
`HX-Trigger-After-Swap`	allows you to trigger client-side events after the swap step

Event Reference

Event	Description
`htmx:abort`	send this event to an element to abort a request

Event	Description
`htmx:afterOnLoad`	triggered after an AJAX request has completed processing a successful response
`htmx:afterProcessNode`	triggered after htmx has initialized a node
`htmx:afterRequest`	triggered after an AJAX request has completed
`htmx:afterSettle`	triggered after the DOM has settled
`htmx:afterSwap`	triggered after new content has been swapped in
`htmx:beforeCleanupElement`	triggered before htmx disables an element or removes it from the DOM
`htmx:beforeOnLoad`	triggered before any response processing occurs
`htmx:beforeProcessNode`	triggered before htmx initializes a node
`htmx:beforeRequest`	triggered before an AJAX request is made
`htmx:beforeSwap`	triggered before a swap is done, allows you to configure the swap
`htmx:beforeSend`	triggered just before an ajax request is sent
`htmx:beforeTransition`	triggered before the View Transition wrapped swap occurs
`htmx:configRequest`	triggered before the request, allows you to customize parameters, headers
`htmx:confirm`	triggered after a trigger occurs on an element, allows you to cancel (or delay) issuing the AJAX request
`htmx:historyCacheError`	triggered on an error during cache writing
`htmx:historyCacheHit`	triggered on a cache hit in the history subsystem
`htmx:historyCacheMiss`	triggered on a cache miss in the history subsystem
`htmx:historyCacheMissLoadError`	triggered on a unsuccessful remote retrieval
`htmx:historyCacheMissLoad`	triggered on a successful remote retrieval
`htmx:historyRestore`	triggered when htmx handles a history restoration action
`htmx:beforeHistorySave`	triggered before content is saved to the history cache
`htmx:load`	triggered when new content is added to the DOM

Event	Description
`htmx:noSSESourceError`	triggered when an element refers to a SSE event in its trigger, but no parent SSE source has been defined
`htmx:onLoadError`	triggered when an exception occurs during the onLoad handling in htmx
`htmx:oobAfterSwap`	triggered after an out of band element as been swapped in
`htmx:oobBeforeSwap`	triggered before an out of band element swap is done, allows you to configure the swap
`htmx:oobErrorNoTarget`	triggered when an out of band element does not have a matching ID in the current DOM
`htmx:prompt`	triggered after a prompt is shown
`htmx:pushedIntoHistory`	triggered after a url is pushed into history
`htmx:replacedInHistory`	triggered after a url is replaced in history
`htmx:responseError`	triggered when an HTTP response error (non- `200` or `300` response code) occurs
`htmx:sendAbort`	triggered when a request is aborted
`htmx:sendError`	triggered when a network error prevents an HTTP request from happening
`htmx:sseError`	triggered when an error occurs with a SSE source
`htmx:sseOpen`	triggered when a SSE source is opened
`htmx:swapError`	triggered when an error occurs during the swap phase
`htmx:targetError`	triggered when an invalid target is specified
`htmx:timeout`	triggered when a request timeout occurs
`htmx:validation:validate`	triggered before an element is validated
`htmx:validation:failed`	triggered when an element fails validation
`htmx:validation:halted`	triggered when a request is halted due to validation errors
`htmx:xhr:abort`	triggered when an ajax request aborts
`htmx:xhr:loadend`	triggered when an ajax request ends

Event	Description
`htmx:xhr:loadstart`	triggered when an ajax request starts
`htmx:xhr:progress`	triggered periodically during an ajax request that supports progress events

JavaScript API Reference

Method	Description
`htmx.addClass()`	Adds a class to the given element
`htmx.ajax()`	Issues an htmx-style ajax request
`htmx.closest()`	Finds the closest parent to the given element matching the selector
`htmx.config`	A property that holds the current htmx config object
`htmx.createEventSource`	A property holding the function to create SSE EventSource objects for htmx
`htmx.createWebSocket`	A property holding the function to create WebSocket objects for htmx
`htmx.defineExtension()`	Defines an htmx extension
`htmx.find()`	Finds a single element matching the selector
`htmx.findAll()` `htmx.findAll(elt, selector)`	Finds all elements matching a given selector
`htmx.logAll()`	Installs a logger that will log all htmx events
`htmx.logger`	A property set to the current logger (default is `null`)
`htmx.off()`	Removes an event listener from the given element
`htmx.on()`	Creates an event listener on the given element, returning it
`htmx.onLoad()`	Adds a callback handler for the `htmx:load` event
`htmx.parseInterval()`	Parses an interval declaration into a millisecond value
`htmx.process()`	Processes the given element and its children, hooking up any htmx behavior
`htmx.remove()`	Removes the given element

Method	Description
`htmx.removeClass()`	Removes a class from the given element
`htmx.removeExtension()`	Removes an htmx extension
`htmx.swap()`	Performs swapping (and settling) of HTML content
`htmx.takeClass()`	Takes a class from other elements for the given element
`htmx.toggleClass()`	Toggles a class from the given element
`htmx.trigger()`	Triggers an event on an element
`htmx.values()`	Returns the input values associated with the given element

Configuration Reference

Htmx has some configuration options that can be accessed either programmatically or declaratively. They are listed below:

Config Variable	Info
`htmx.config.historyEnabled`	defaults to `true`, really only useful for testing
`htmx.config.historyCacheSize`	defaults to 10
`htmx.config.refreshOnHistoryMiss`	defaults to `false`, if set to `true` htmx will issue a full page refresh on history misses rather than use an AJAX request
`htmx.config.defaultSwapStyle`	defaults to `innerHTML`
`htmx.config.defaultSwapDelay`	defaults to 0
`htmx.config.defaultSettleDelay`	defaults to 20
`htmx.config.includeIndicatorStyles`	defaults to `true` (determines if the indicator styles are loaded)
`htmx.config.indicatorClass`	defaults to `htmx-indicator`
`htmx.config.requestClass`	defaults to `htmx-request`
`htmx.config.addedClass`	defaults to `htmx-added`
`htmx.config.settlingClass`	defaults to `htmx-settling`

Config Variable	Info
`htmx.config.swappingClass`	defaults to `htmx-swapping`
`htmx.config.allowEval`	defaults to `true` , can be used to disable htmx's use of eval for certain features (e.g. trigger filters)
`htmx.config.allowScriptTags`	defaults to `true` , determines if htmx will process script tags found in new content
`htmx.config.inlineScriptNonce`	defaults to `''` , meaning that no nonce will be added to inline scripts
`htmx.config.inlineStyleNonce`	defaults to `''` , meaning that no nonce will be added to inline styles
`htmx.config.attributesToSettle`	defaults to `["class", "style", "width", "height"]` , the attributes to settle during the settling phase
`htmx.config.wsReconnectDelay`	defaults to `full-jitter`
`htmx.config.wsBinaryType`	defaults to `blob` , the the type of binary data being received over the WebSocket connection
`htmx.config.disableSelector`	defaults to `[hx-disable], [data-hx-disable]` , htmx will not process elements with this attribute on it or a parent
`htmx.config.disableInheritance`	defaults to `false` . If it is set to `true` , the inheritance of attributes is completely disabled and you can explicitly specify the inheritance with the hx-inherit attribute.
`htmx.config.withCredentials`	defaults to `false` , allow cross-site Access-Control requests using credentials such as cookies, authorization headers or TLS client certificates
`htmx.config.timeout`	defaults to 0, the number of milliseconds a request can take before automatically being terminated
`htmx.config.scrollBehavior`	defaults to 'instant', the scroll behavior when using the show modifier with `hx-swap` . The allowed values are `instant` (scrolling should happen instantly in a

Config Variable	Info
	single jump), `smooth` (scrolling should animate smoothly) and `auto` (scroll behavior is determined by the computed value of scroll-behavior).
`htmx.config.defaultFocusScroll`	if the focused element should be scrolled into view, defaults to false and can be overridden using the focus-scroll swap modifier.
`htmx.config.getCacheBusterParam`	defaults to false, if set to true htmx will append the target element to the `GET` request in the format `org.htmx.cache-buster=targetElementId`
`htmx.config.globalViewTransitions`	if set to `true`, htmx will use the View Transition API when swapping in new content.
`htmx.config.methodsThatUseUrlParams`	defaults to `["get", "delete"]`, htmx will format requests with these methods by encoding their parameters in the URL, not the request body
`htmx.config.selfRequestsOnly`	defaults to `true`, whether to only allow AJAX requests to the same domain as the current document
`htmx.config.ignoreTitle`	defaults to `false`, if set to `true` htmx will not update the title of the document when a `title` tag is found in new content
`htmx.config.scrollIntoViewOnBoost`	defaults to `true`, whether or not the target of a boosted element is scrolled into the viewport. If `hx-target` is omitted on a boosted element, the target defaults to `body`, causing the page to scroll to the top.
`htmx.config.triggerSpecsCache`	defaults to `null`, the cache to store evaluated trigger specifications into, improving parsing performance at the cost of more memory usage. You may define a simple object to use a never-clearing

Config Variable	Info
	cache, or implement your own system using a proxy object
`htmx.config.responseHandling`	the default Response Handling behavior for response status codes can be configured here to either swap or error
`htmx.config.allowNestedOobSwaps`	defaults to `true`, whether to process OOB swaps on elements that are nested within the main response element. See Nested OOB Swaps.
`htmx.config.historyRestoreAsHxRequest`	defaults to `true`, Whether to treat history cache miss full page reload requests as a "HX-Request" by returning this response header. This should always be disabled when using HX-Request header to optionally return partial responses
`htmx.config.reportValidityOfForms`	defaults to `false`, Whether to report input validation errors to the end user and update focus to the first input that fails validation. This should always be enabled as this matches default browser form submit behaviour

You can set them directly in javascript, or you can use a `meta` tag:

```
<meta name="htmx-config" content='{"defaultSwapStyle":"outerHTML"}'>
```

haiku

javascript fatigue:
longing for a hypertext
already in hand

---

docs
reference

examples
talk
essays
@htmx_org